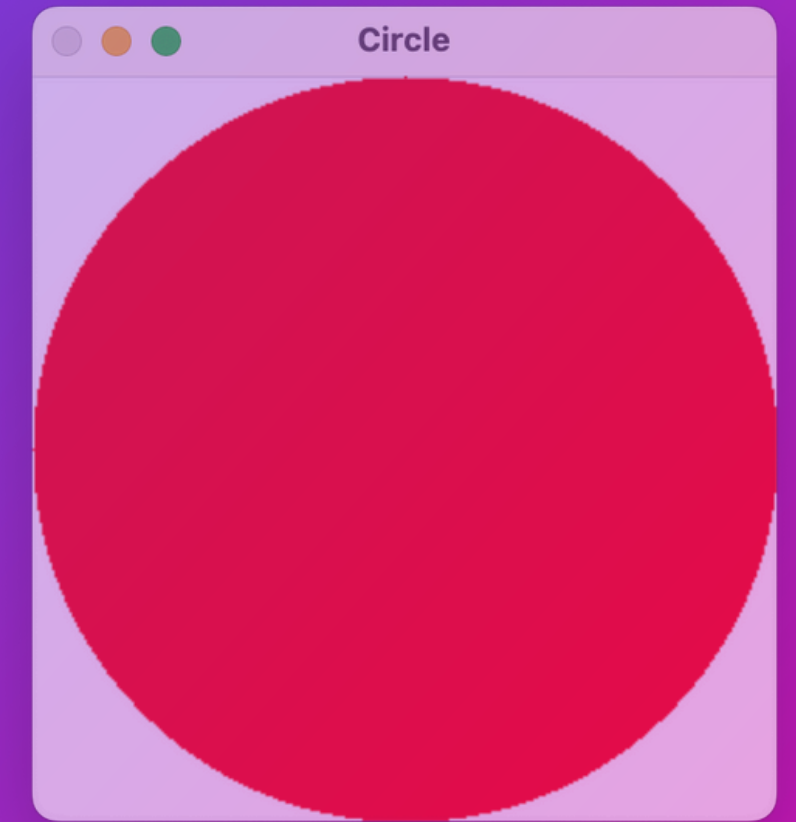


Bitwise Mask inrange

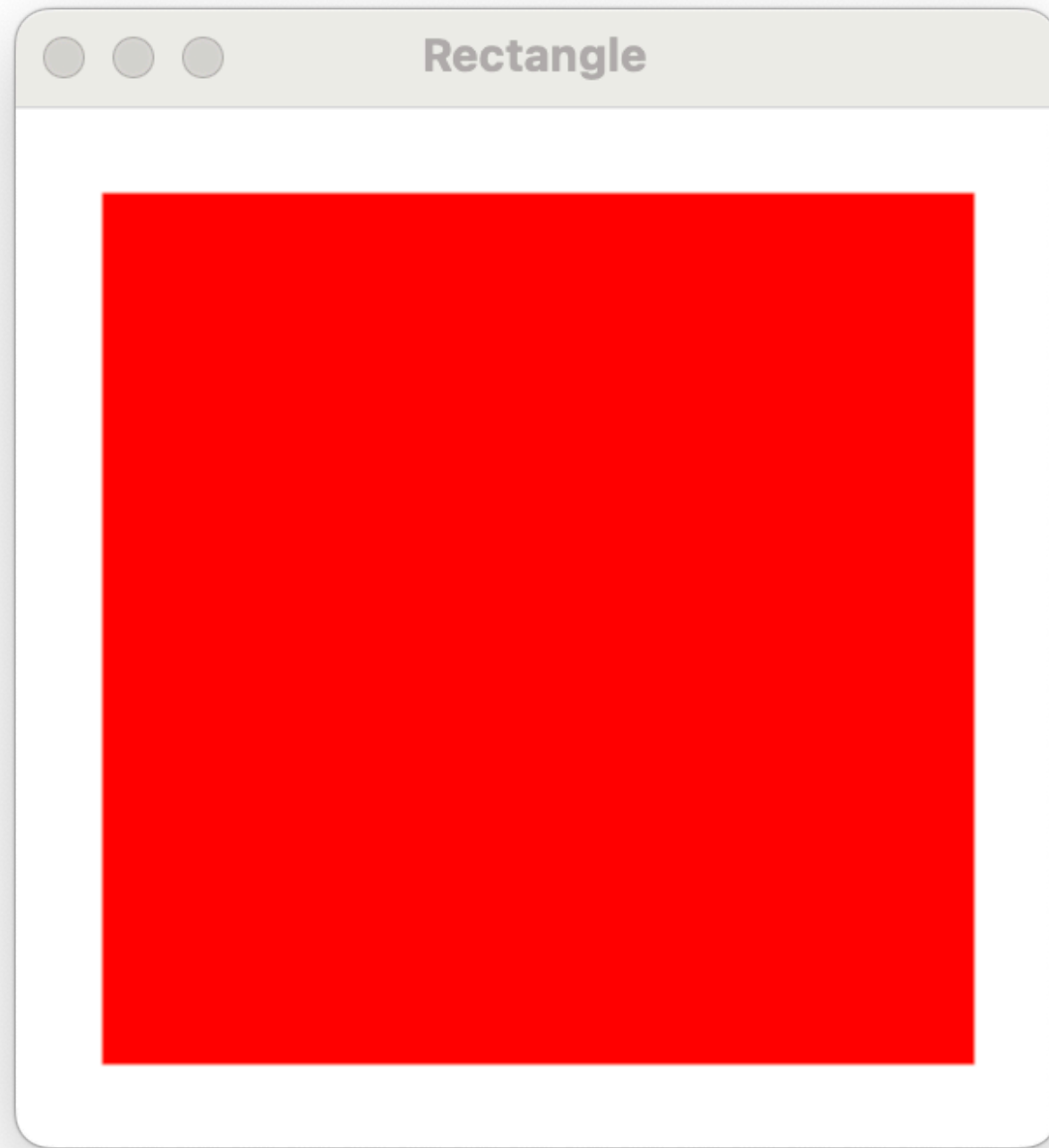


BITWISE OPENCV

เป็นตัวดำเนินการในระดับบิตกับรูปภาพโดยใช้ OpenCV โดยคำสั่ง bitwise_ โดย
รายละเอียดคำสั่งทั้ง 4 คือ

1. **bitwise_and**
2. **bitwise_or**
3. **bitwise_not**
4. **bitwise_xor**

OUTPUT



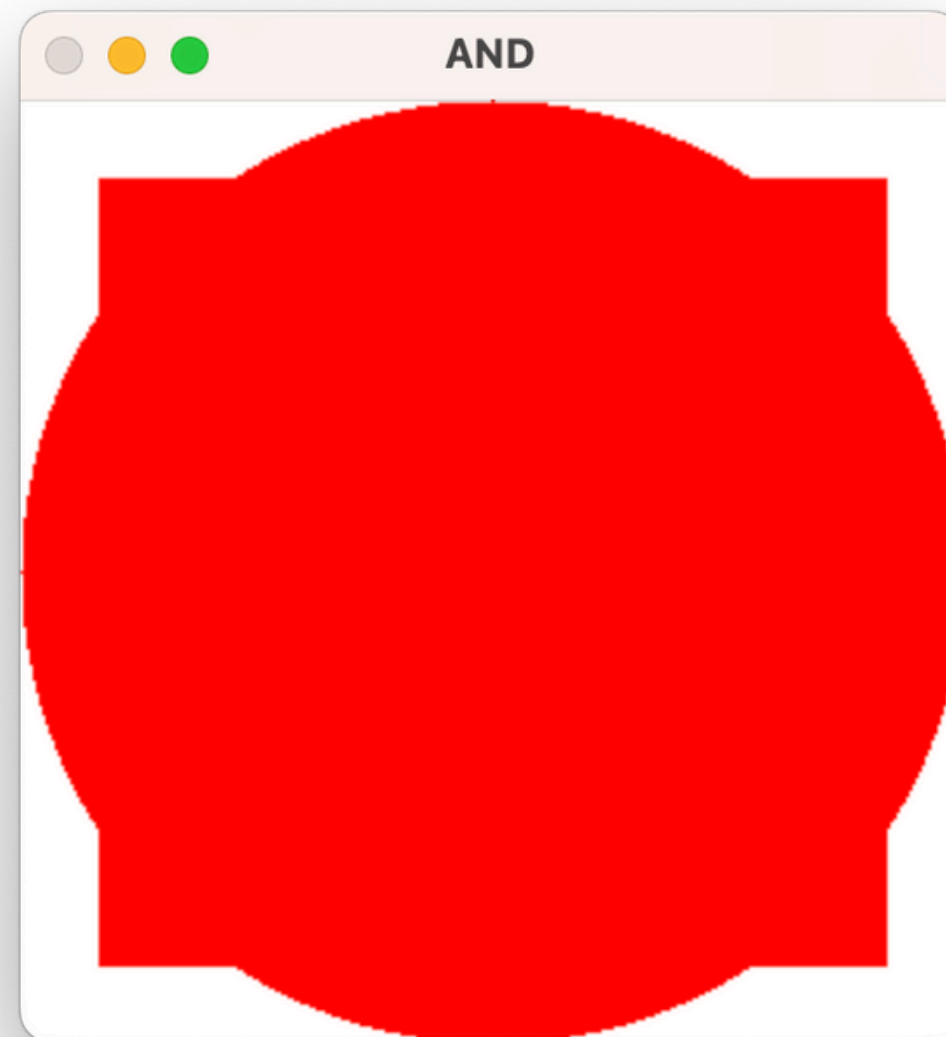
CODE

```
1 import numpy as np
2 import cv2 as cv
3
4 rectangle = np.zeros((300, 300,3), dtype="uint8")
5 rectangle[:] = [255,255,255]
6 cv.rectangle(rectangle, (25, 25), (275, 275), (0, 0, 255), -1)
7 cv.imshow("Rectangle", rectangle)
8
9 circle = np.zeros((300, 300,3), dtype = "uint8")
10 circle[:] = [255,255,255]
11 cv.circle(circle, (150, 150), 150, (0, 0, 255), -1)
12 cv.imshow("Circle", circle)
```

BITWISE_AND

- **bitwise_and(img1, img2 mask=mask)** หมายถึง ภาพ img1 และ img2 สองภาพเหมือนกันรวมกัน
- **Mask** คือหน้ากาก หรือ ส่วนที่ปิด บัง ไว้ซึ่งในที่นี้ คือการเอาเฉพาะส่วนที่ ไม่ได้ถูกบังไว้ของภาพที่ 1 และ 2 มารวมกับ mask

OUTPUT



CODE



```
1 bitwise_and = cv.bitwise_and(rectangle,circle)
2 cv.imshow("AND", bitwise_and)
```

BITWISE_OR

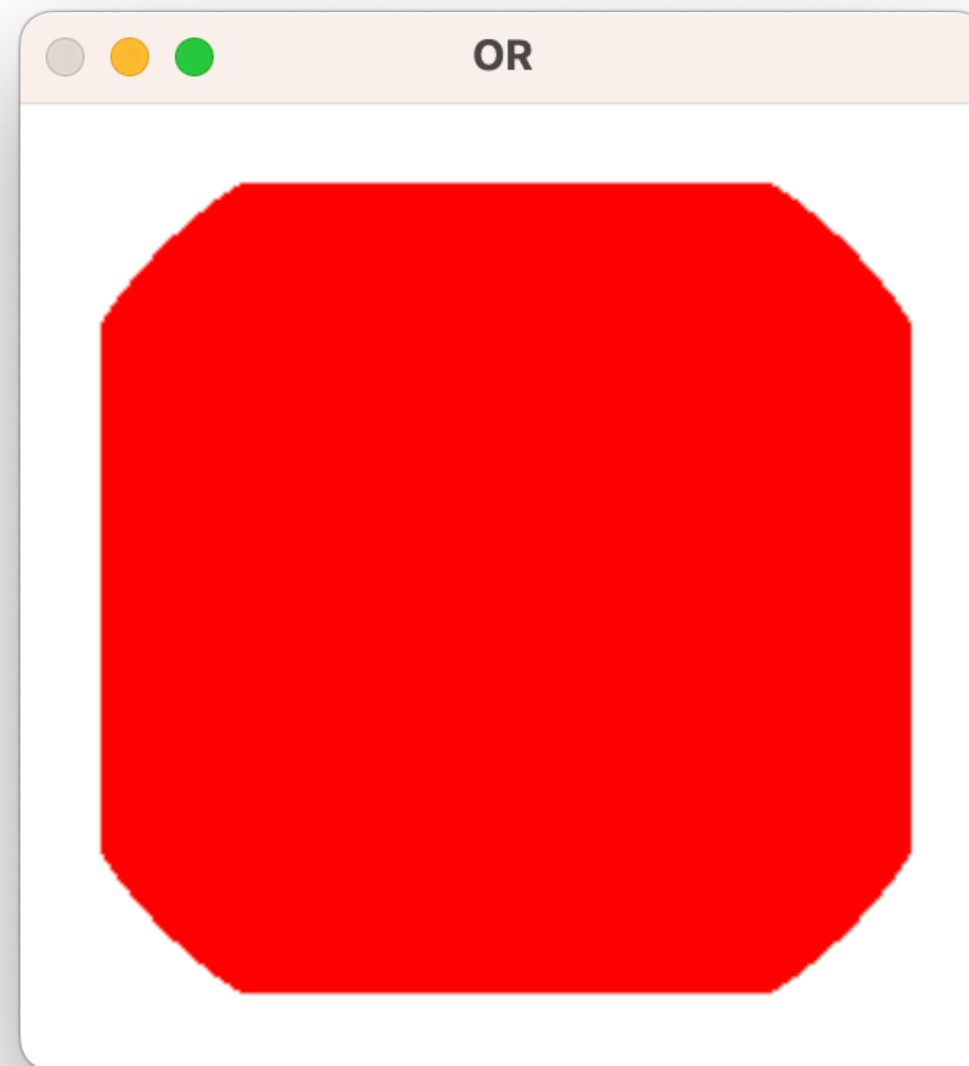
- **bitwise_or(img1, img2)** หมายถึง การแสดงภาพเฉพาะส่วนที่เหมือนกันของ **img1** และ **img2**

CODE



```
1 bitwise_or = cv.bitwise_or(rectangle,circle)
2 cv.imshow("OR", bitwise_or)
```

OUTPUT



BITWISE_NOT

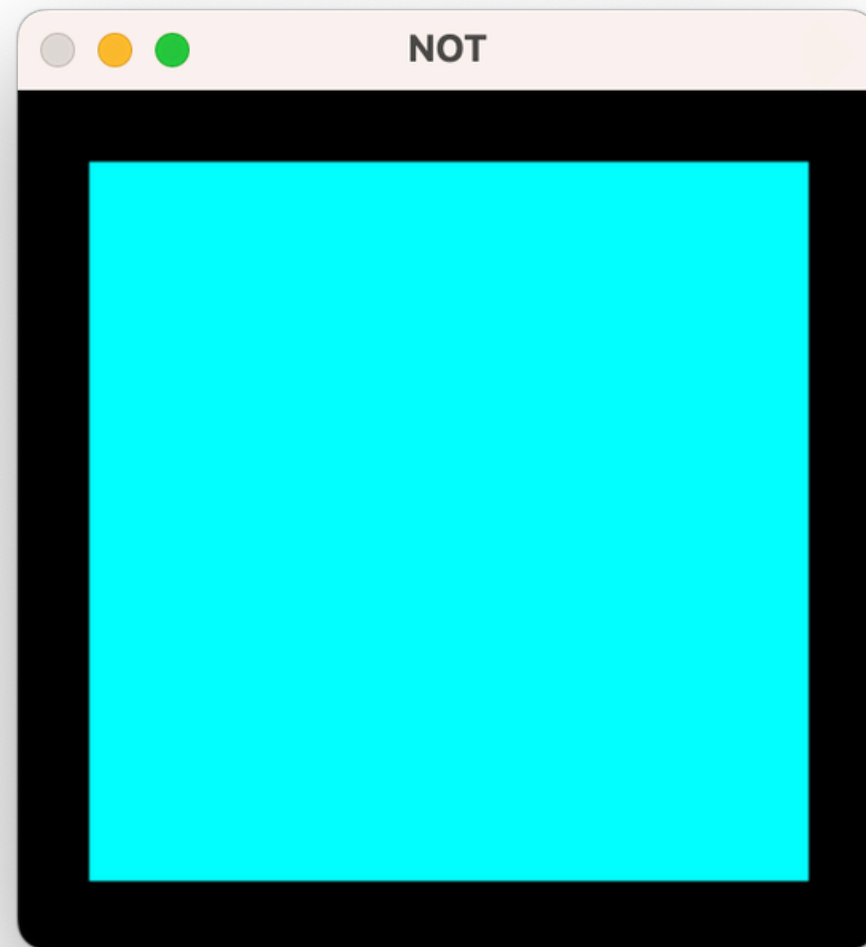
bitwise_not(img1) หมายถึง
ถึง การแสดงภาพส่วนกลับ
ของรูปที่ 1

CODE



```
1 bitwise_not = cv.bitwise_not(rectangle)
2 cv.imshow("NOT", bitwise_not)
```

OUTPUT



BITWISE_XOR

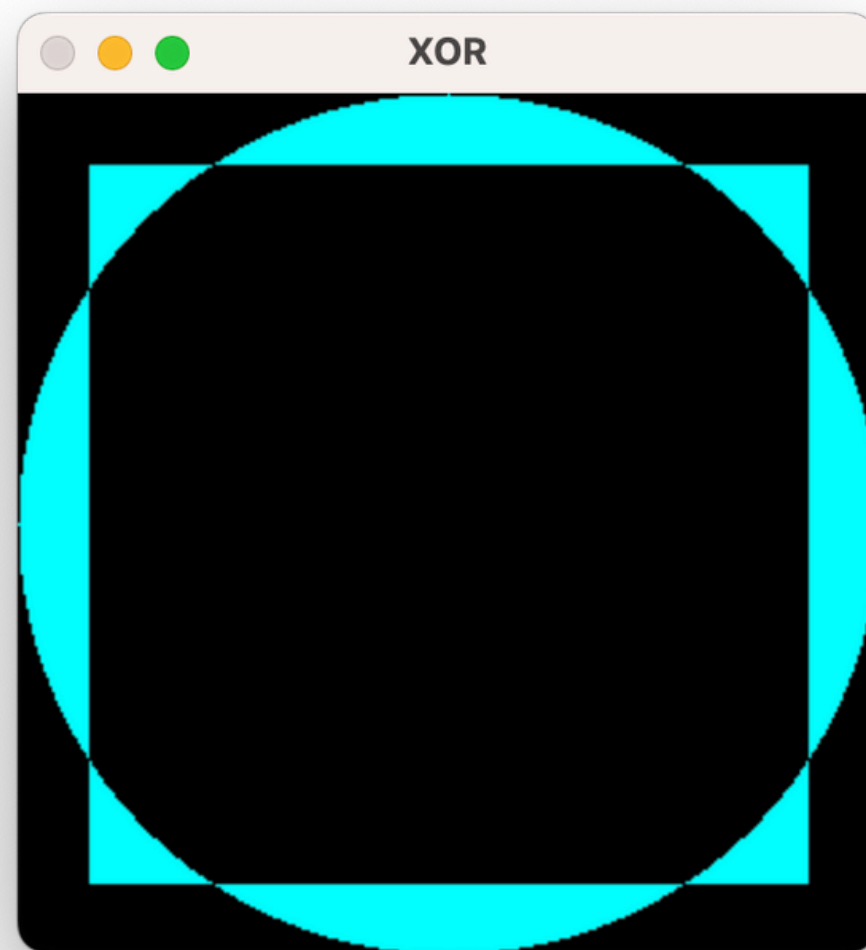
bitwise_xor(img1, img2) หมายถึง
ถึง การแสดงภาพส่วนที่ไม่เหมือนกัน
ของ **img1** และ **img2**

CODE



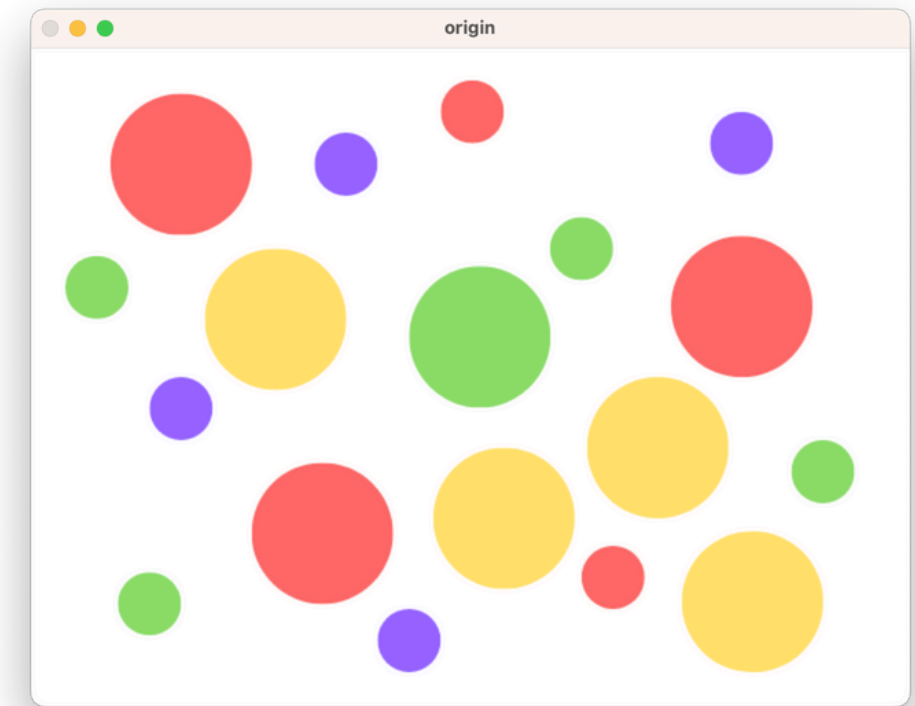
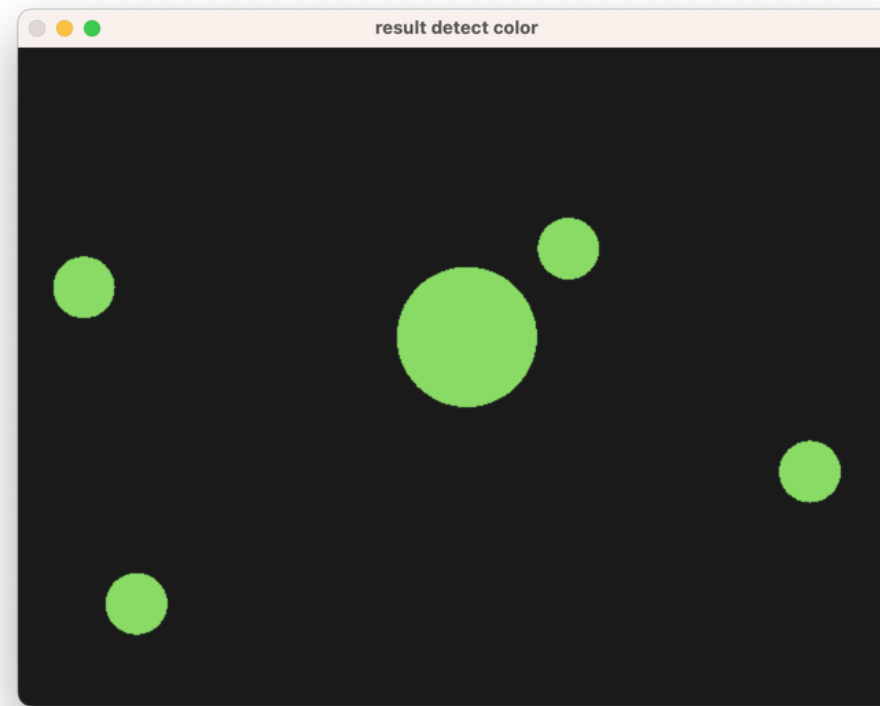
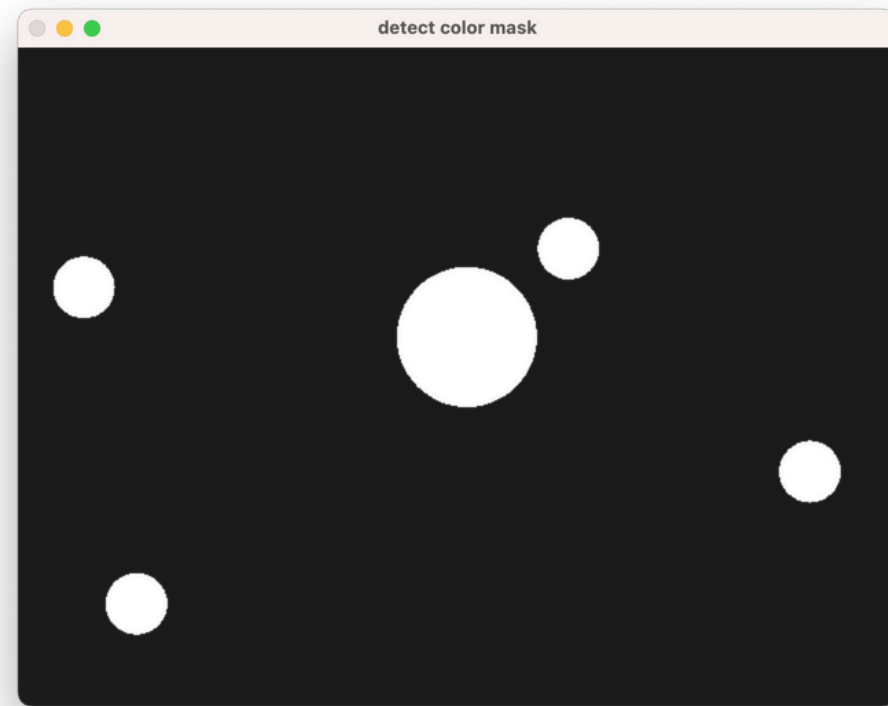
```
1 bitwise_xor = cv.bitwise_xor(rectangle, circle)
2 cv.imshow("XOR", bitwise_xor)
```

OUTPUT



Detect color

การตรวจจับกลุ่มวัตถุ ด้วยสี

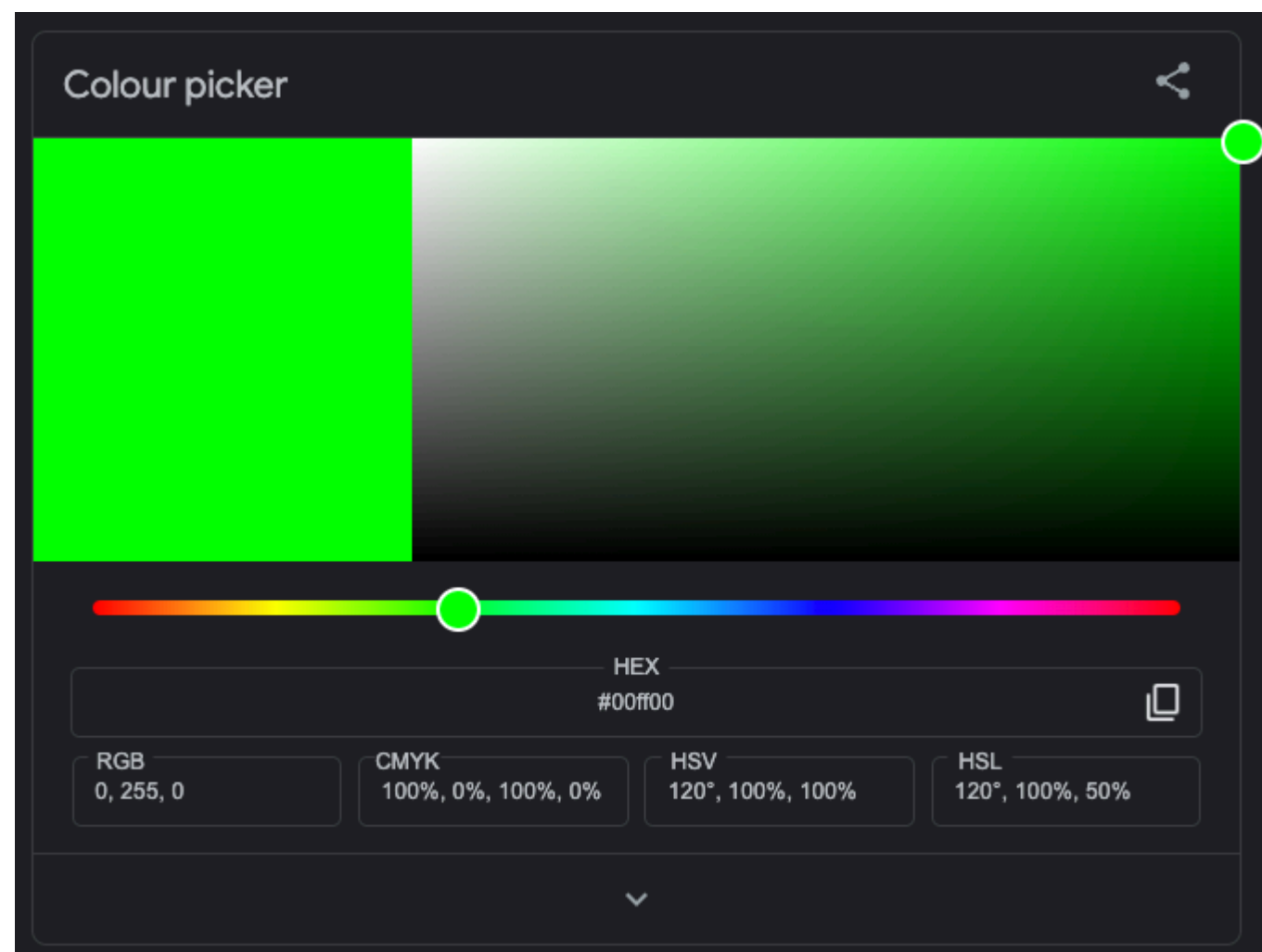


Detect color

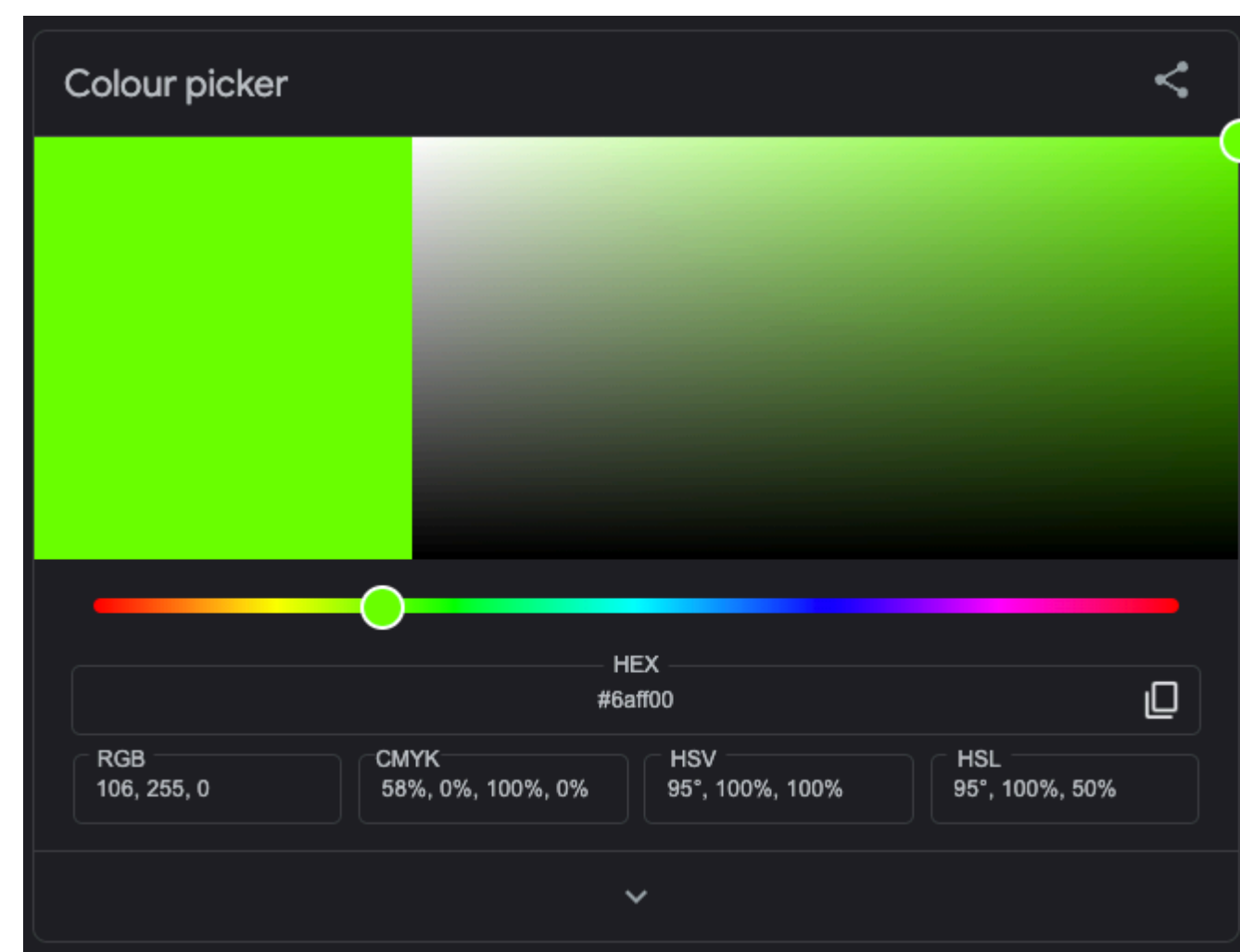
Detect color

หลักการทำงานคือ ระบุช่วงของสีต่างๆ เช่น
สีเขียว ซึ่งสีเขียวมาตรฐาน จะมีค่าสี BGR คือ (0,255,0) แต่สี
เขียวนั้นมีสีเขียวหลายแบบ เช่น เขียวอมม่วง สีเขียวอมแดง สี
เขียวอมดำ

GREEN

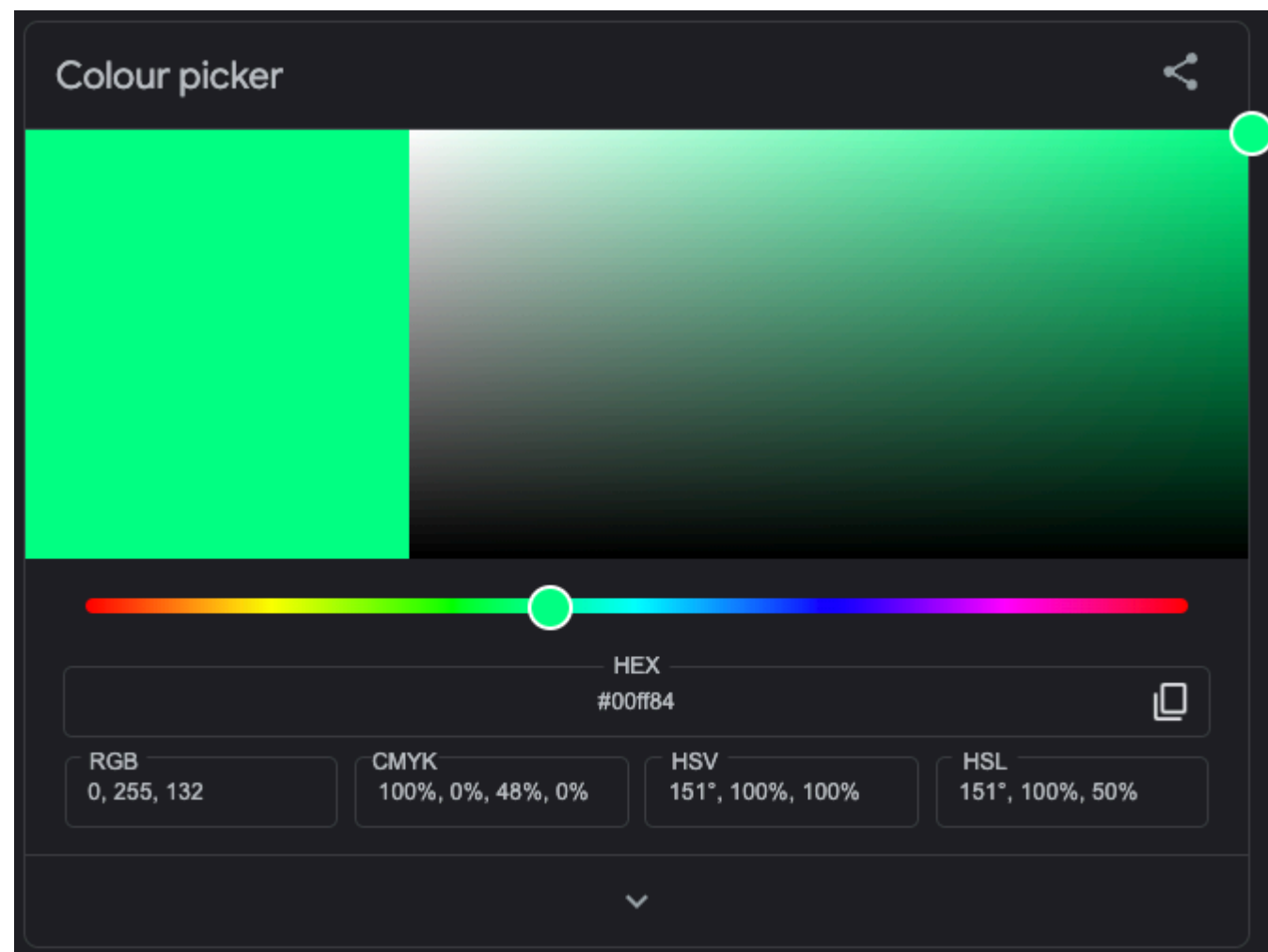


สีเขียวมาตรฐาน BGR(0,255,0)

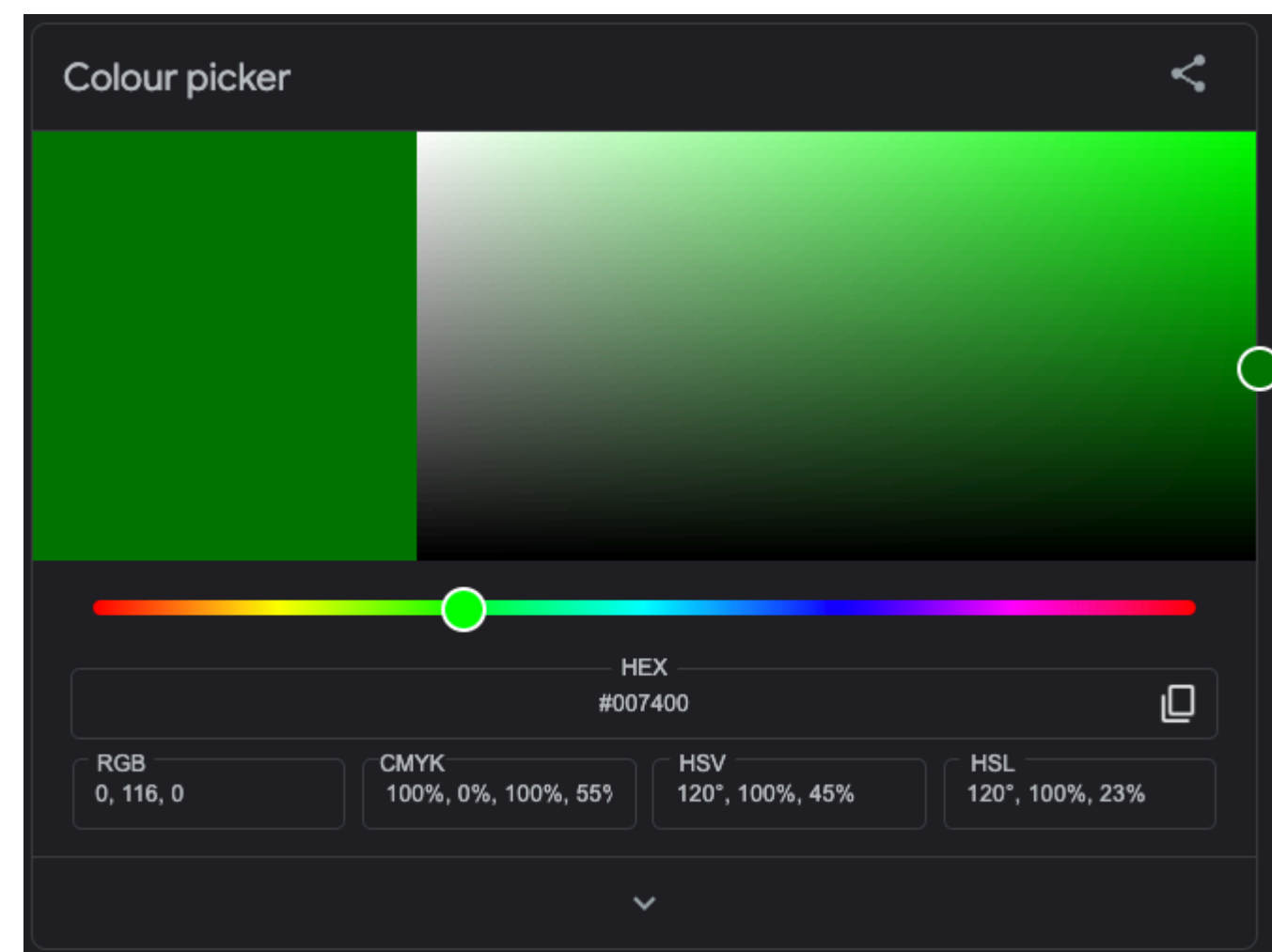


สีเขียวอมแดง BGR(0,214,106)

GREEN



สีเขียวอมน้ำเงิน BGR(132,255,0)

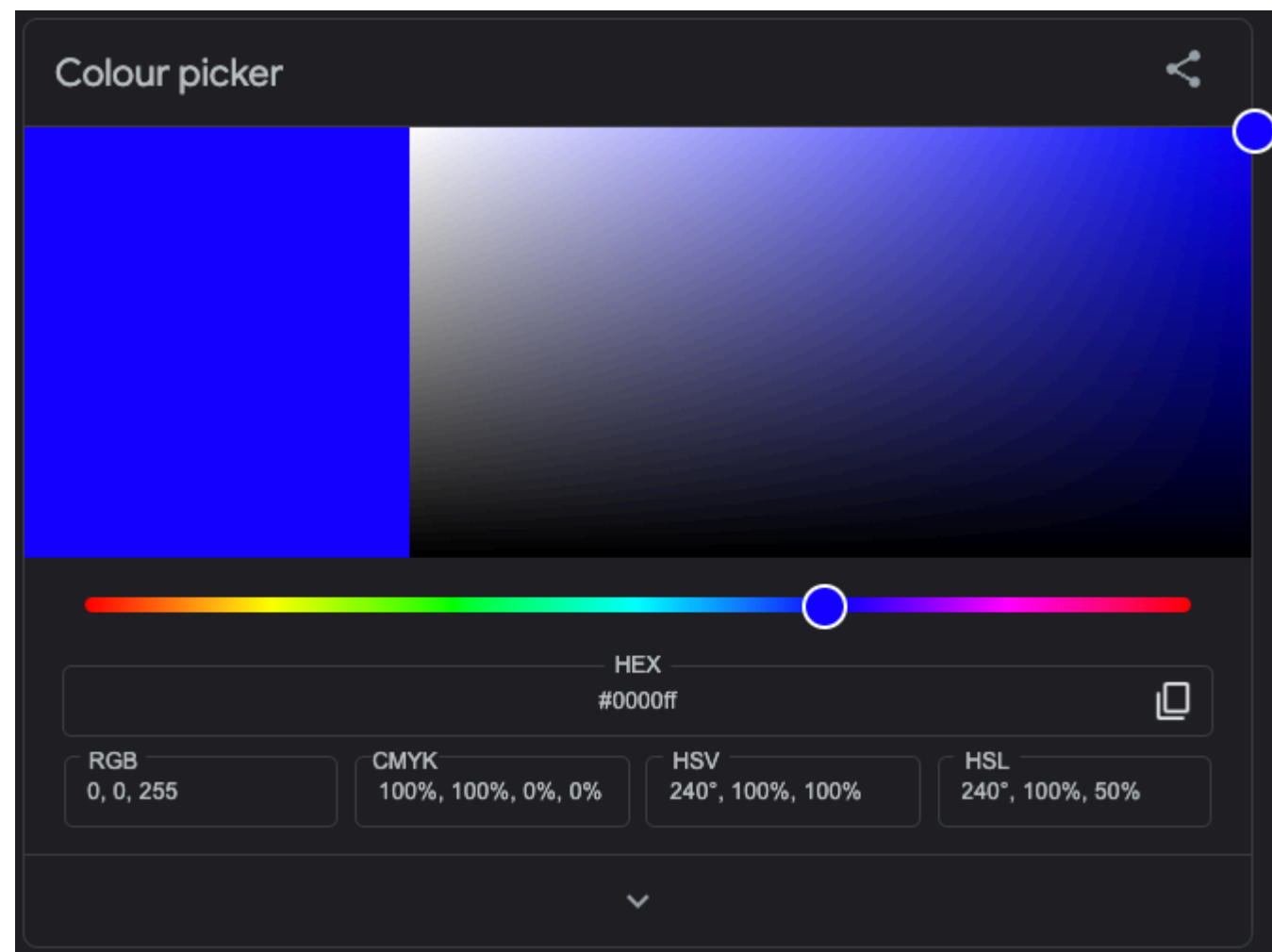


สีเขียวอมดำ BGR(0,116,0)

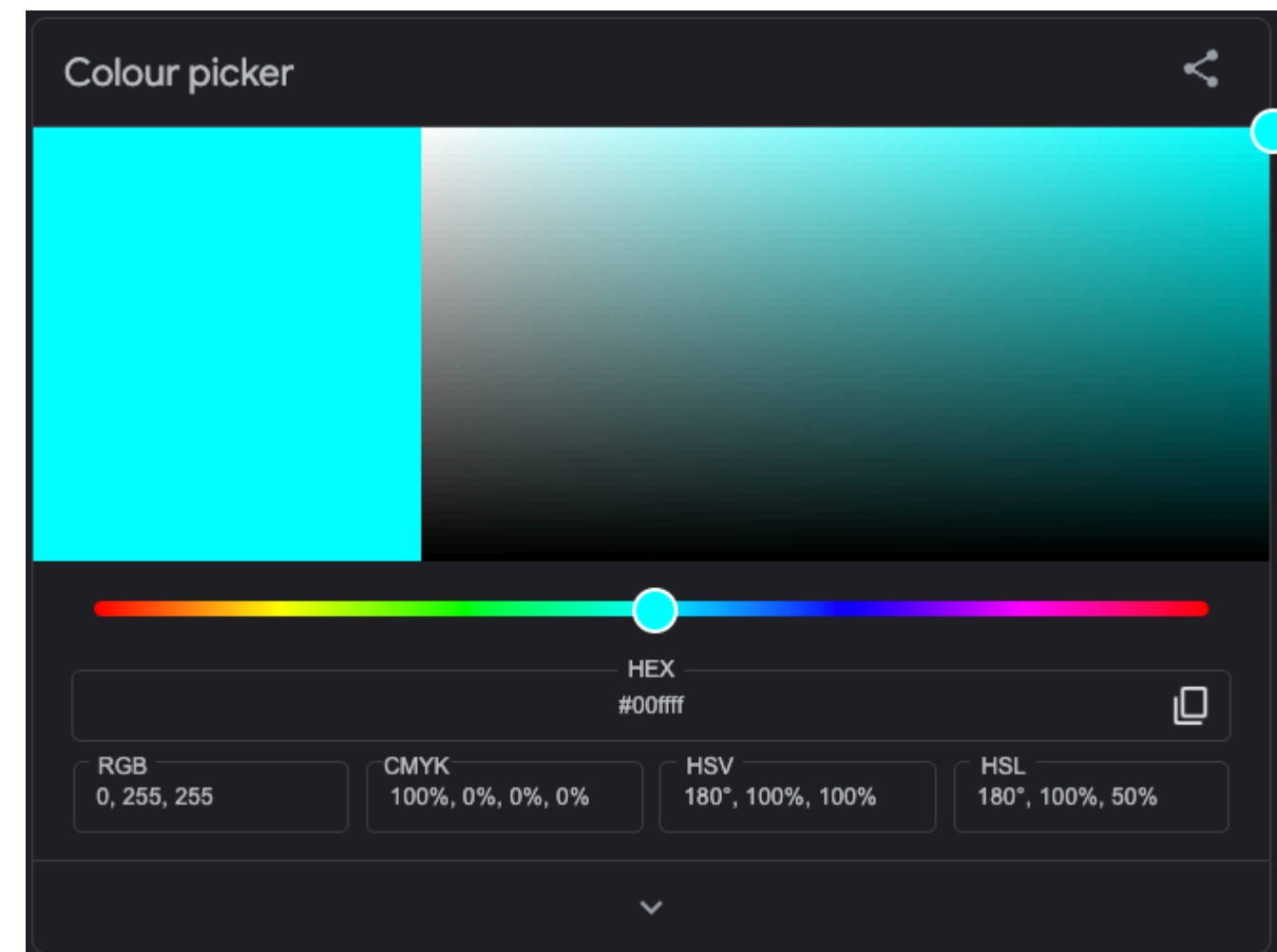
GREEN

- ดังนั้นการกำหนดหาช่วงของสีเขียวนั้น อาจจะกำหนดตามช่วงดังกล่าวได้โดยการกำหนด
 - ค่าช่วง สีเขียวสูงสุดที่อาจเป็นไปได้ ซึ่งจะได้ค่าเป็น `upper_green = (132,255,106)`
 - ค่าช่วงสีเขียวต่ำสุดที่อาจเป็นไปได้ ซึ่งจะได้ค่าเป็น `lower_green = (0,116,0)`
- ซึ่งการกำหนดช่วงของสีนั้น เพื่อนำมาทำการ `mask` หรือการครอบสีที่อยู่ในช่วงนั้นไว้และให้สีที่ไม่ได้ในช่วงสีที่ระบุกลายเป็นสีดำ

BLUE

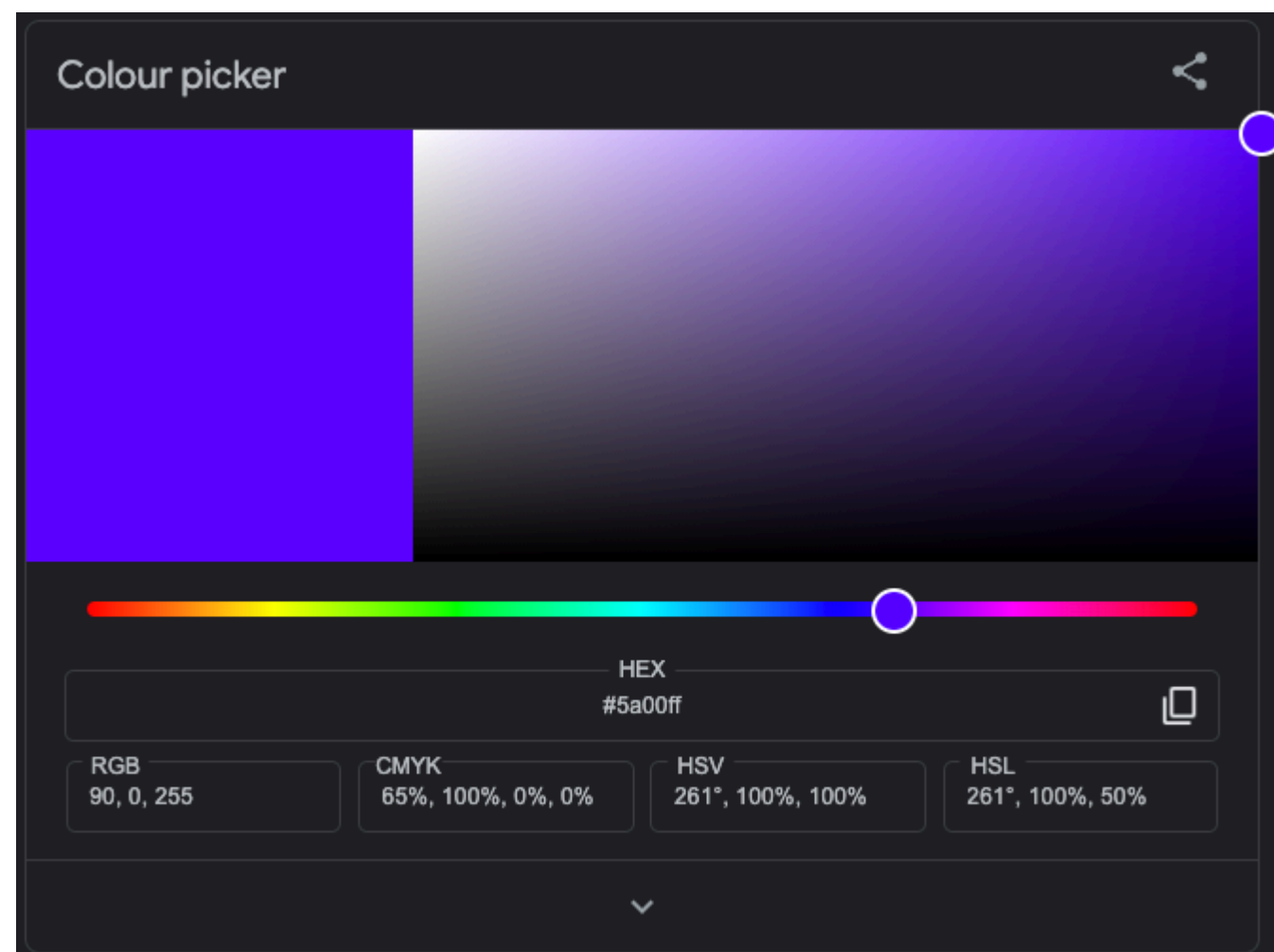


สีน้ำเงินมาตรฐาน BGR (255,0,0)

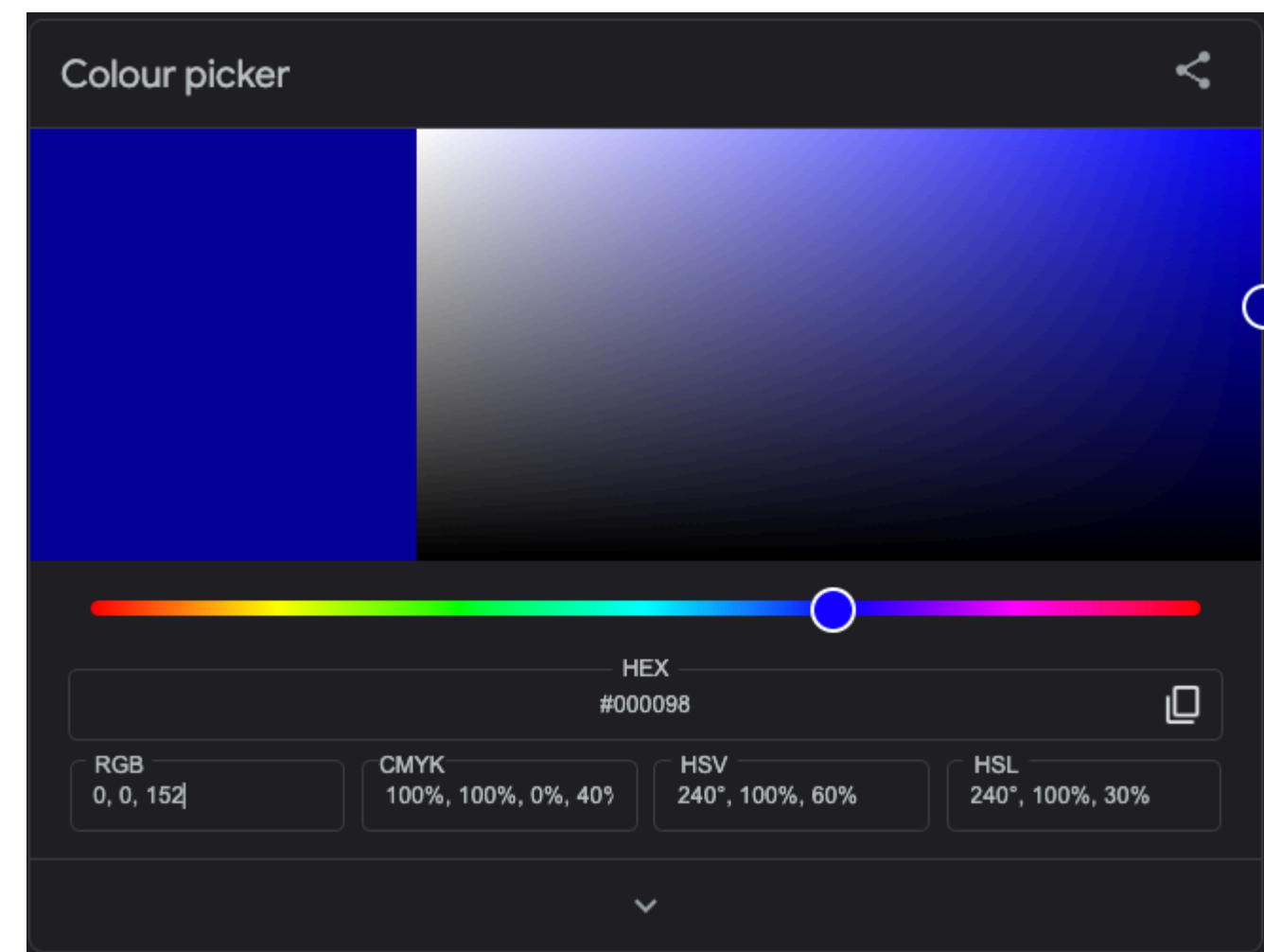


สีน้ำเงินอมเขียว BGR (255,255,0)

BLUE



สีน้ำเงินอมแดง BGR (0,255,90)

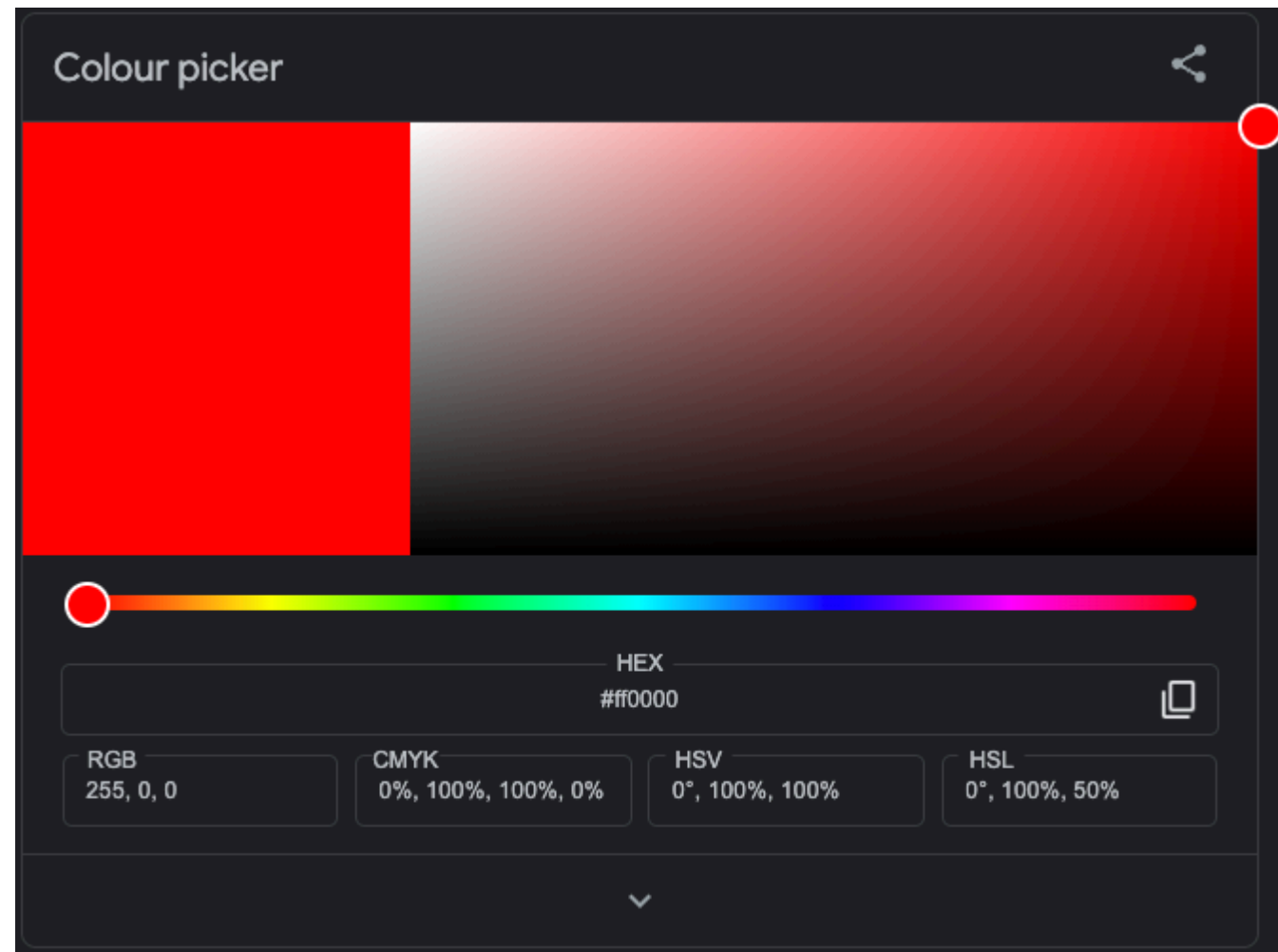


สีน้ำเงินอมดำ BGR (152,0,0)

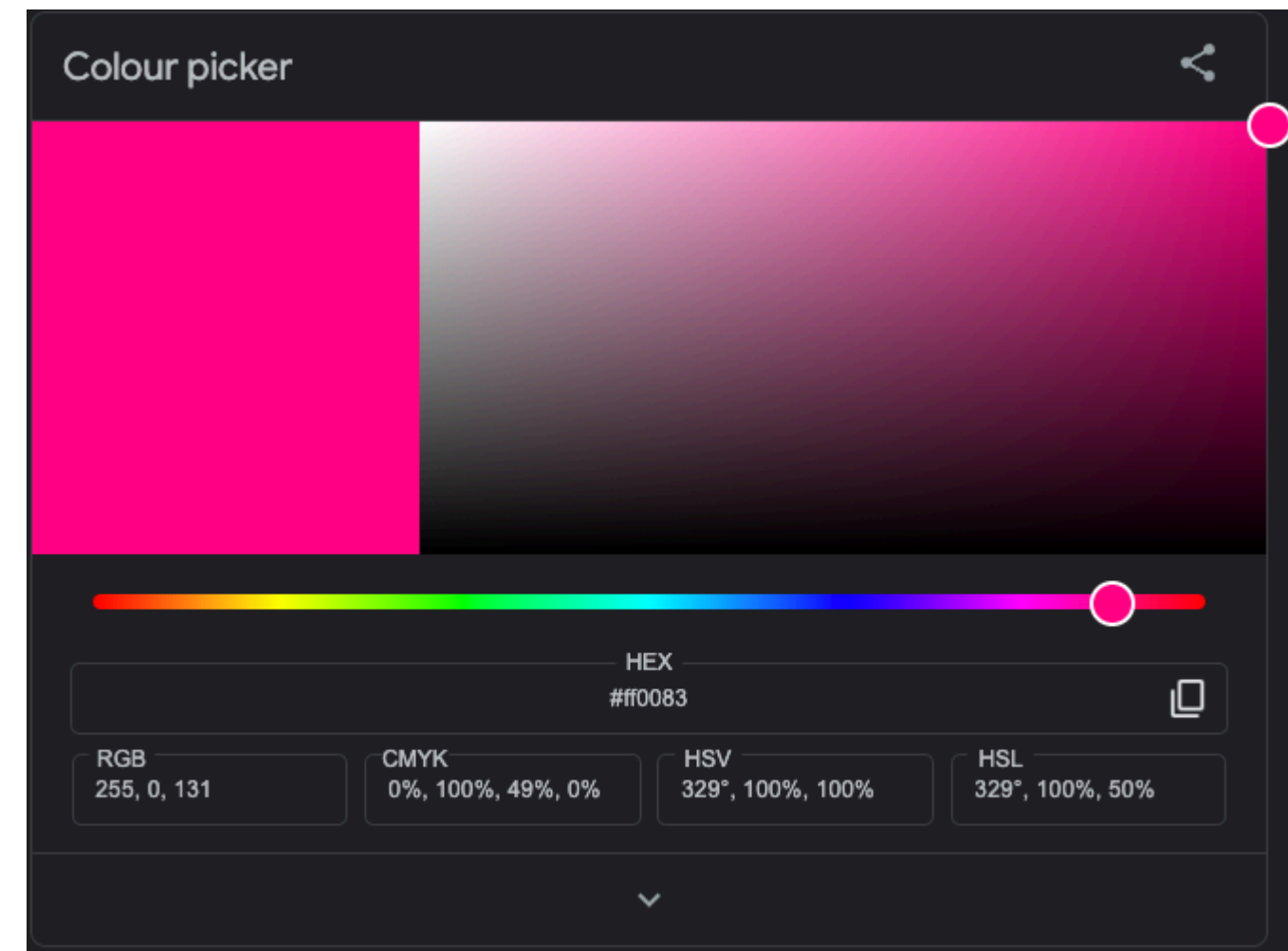
BLUE

- ดังนั้นการกำหนดหาช่วงของสีน้ำเงินนั้น อาจจะกำหนดตามช่วงดังกล่าวได้โดยการกำหนด
- ค่าช่วง สีน้ำเงินสูงสุดที่อาจเป็นไปได้ ซึ่งจะได้ค่าเป็น `upper_blue = (255,255,90)`
- ค่าช่วงสีน้ำเงินต่ำสุดที่อาจเป็นไปได้ ซึ่งจะได้ค่าเป็น `lower_blue = (152,0,0)`

RED

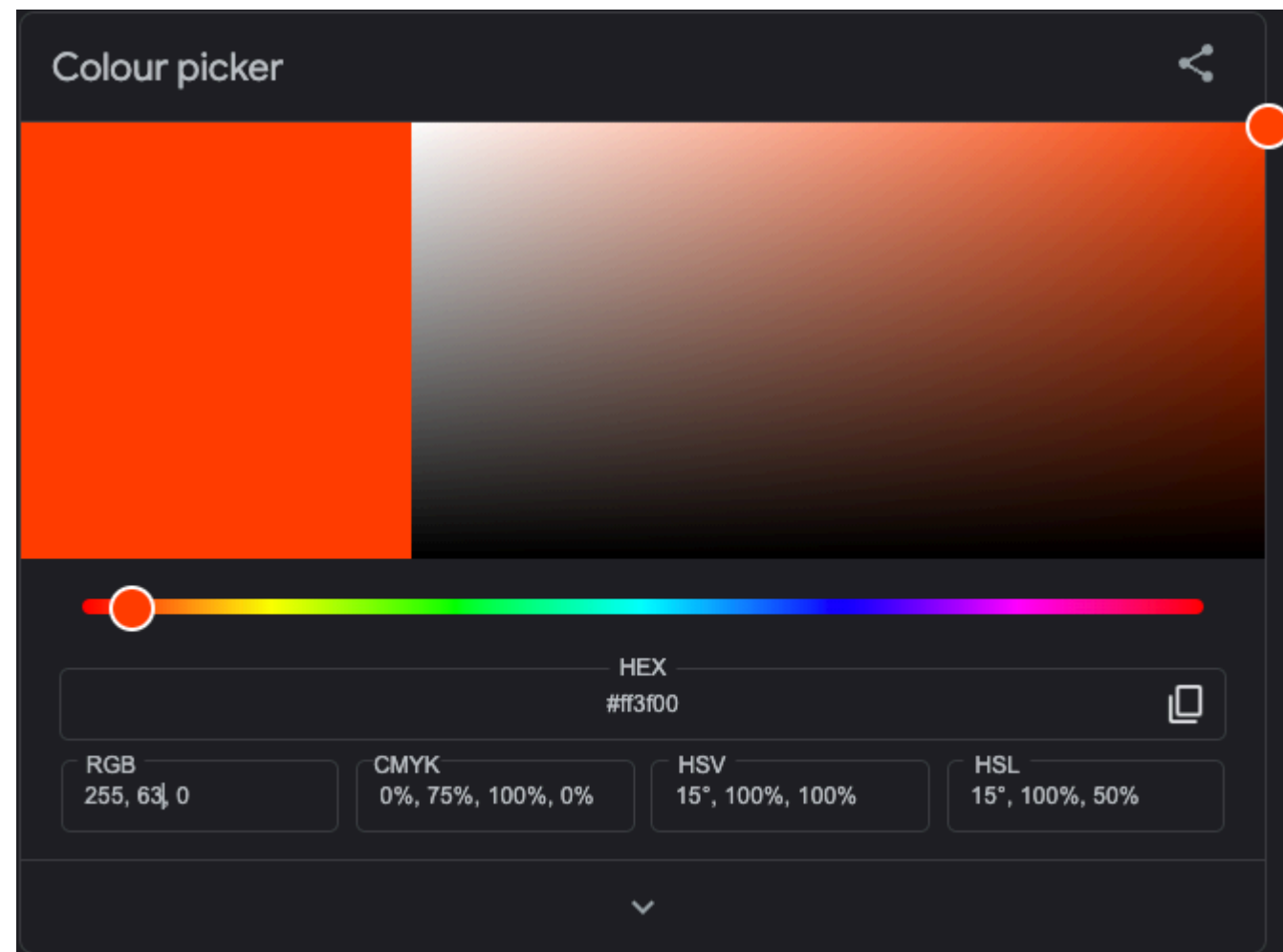


สีแดงมาตรฐาน BGR (0,0,255)

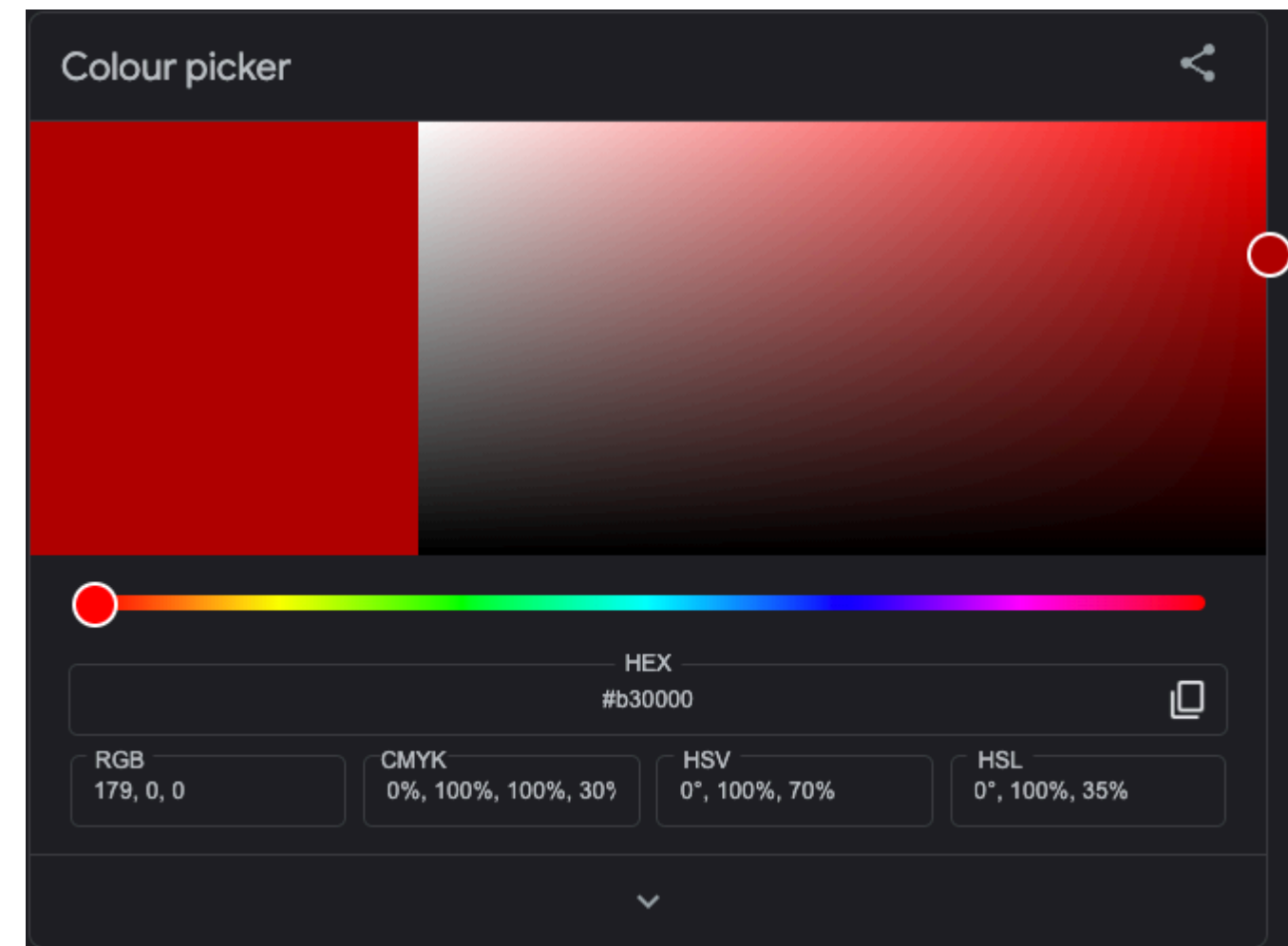


สีแดงอมมagenta BGR (131,0,255)

RED



สีแดงอมเขียว BGR (0,63,255)



สีแดงอมดำ BGR (0,0,179)

RED

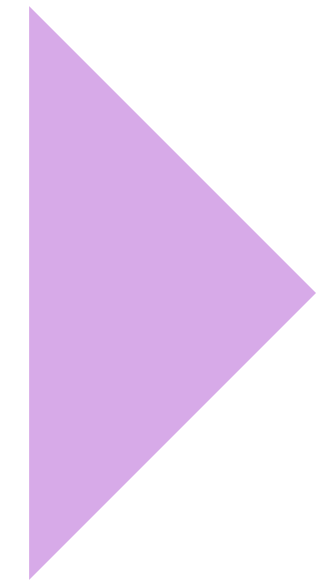
- ดังนั้นการกำหนดหาช่วงของสีแดงนั้น อาจจะกำหนดตามช่วงดังกล่าวได้โดยการกำหนด
- ค่าช่วง สีน้ำเงินสูงสุดที่อาจเป็นไปได้ ซึ่งจะได้ค่าเป็น `upper_red = (131,63,255)`
- ค่าช่วงสีน้ำเงินต่ำสุดที่อาจเป็นไปได้ ซึ่งจะได้ค่าเป็น `lower_red = (0,0,179)`

DETECT OBJECT

นำค่าที่กำหนดช่วงของค่ามาทำการ mask ภาพโดยใช้คำสั่ง `cv.inRange()` โดยนำค่าต่ำสุด และสูงสุดมาใส่เพื่อจะทำการ mask หรือ ตัดภาพสีนั้นออกไป

CODE

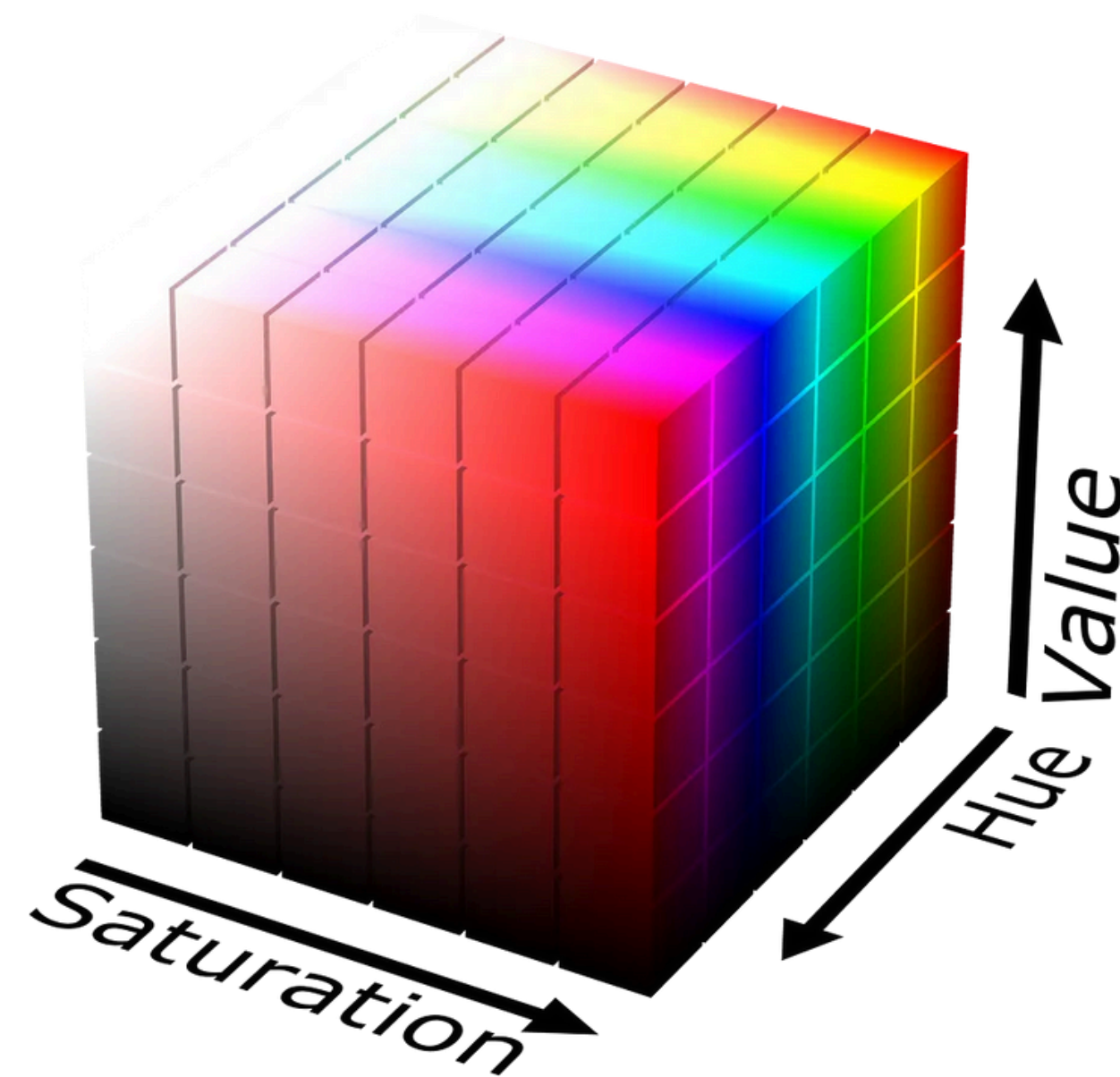
```
● ● ●  
1 import cv2 as cv  
2 import numpy as np  
3  
4 img = cv.imread("image/lunares.png")  
5 upper_green = np.array([100, 255, 130])  
6 lower_green = np.array([0, 50, 0])  
7 mask = cv.inRange(img, lower_green, upper_green)  
8 result = cv.bitwise_and(img, img, mask=mask)  
9  
10 cv.imshow("origin", img)  
11 cv.imshow("detect color mask", mask)  
12 cv.imshow("result detect color", result)  
13 cv.waitKey(0)  
14 cv.destroyAllWindows()
```



HSV Color

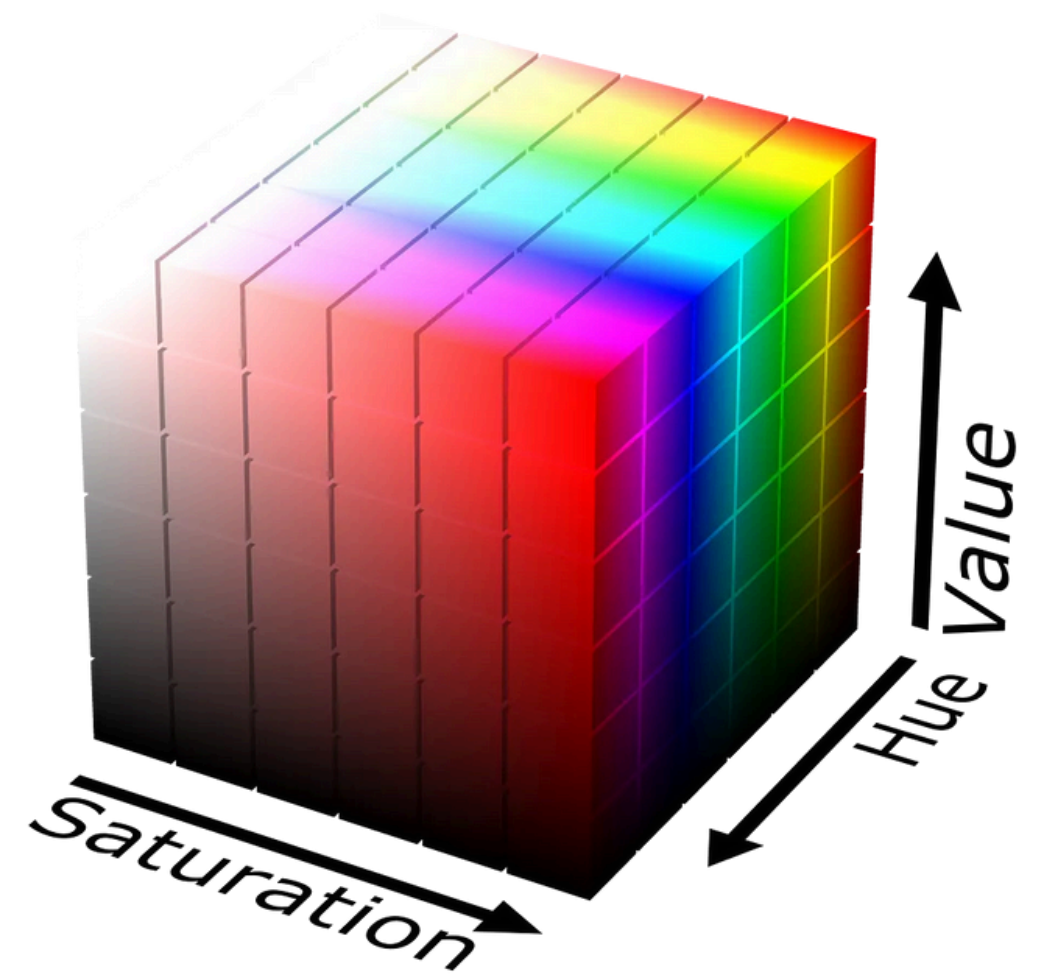
HSV filter

ระบบสี HSV (Hue , Saturation , Value) เป็นระบบสีและการแสดงสีที่ใช้กันอย่างแพร่หลายในการปรับแต่งสีของภาพ โดยที่ความหมายของแต่ละส่วน คือ ระบบสี HSV (Hue , Saturation , Value) เป็นระบบสีและการแสดงสีที่ใช้กันอย่างแพร่หลายในการปรับแต่งสีของภาพ โดยที่ความหมายของแต่ละส่วน



HSV filter

1. Hue : เป็นค่าที่จะบอกสีหรือโทนสี ที่เรามองเห็น
เช่น สีแดง สีม่วง สีเขียว
2. Saturation : เป็นค่าที่บอกถึงความสดของสีหรือ
ความอิ่มตัวของสี ค่าที่สูงก็หมายถึงสีสดและมี
ความเข้ม(Saturation) ในขณะที่ค่าที่ต่ำก็หมายถึง
สีซีดจางหรือสีเทา
3. Value : เป็นค่าที่บอกถึงระดับของความสว่างของ
สี ค่าที่สูงก็หมายถึงสีที่มีความเข้ม(Saturation)
และค่าที่ต่ำก็หมายถึงสีที่สว่างน้อย



HSV filter

เปลี่ยนภาพที่ได้จากกล้อง จากระบบสี RGB เป็นระบบสี HSV

```
32  
33     roi = frame[10:450, 180:400]  
34     hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)  
35
```

HSV filter

หาช่วงค่า

```
5
6  lower_red = np.array([0,88,210])
7  upper_red = np.array([15,242,255])
8
9  lower_blue = np.array([83,36,155])
10 upper_blue = np.array([134,255,255])
11
12 lower_green = np.array([45,38,80])
13 upper_green = np.array([76,255,255])
14
15 lower_yellow = np.array([10,86,200])
16 upper_yellow = np.array([84,170,255])
17
18 List_Colors = [(lower_red, upper_red),
19                (lower_blue, upper_blue),
20                (lower_green, upper_green),
21                (lower_yellow, upper_yellow)]
22
23 str_color = ["Red", "Blue", "Green", "Yellow"]
24
```